

# Report — Clothes Segmentation System (CVCY002)

**Position:** Computer Vision Engineer    **Objective:** Develop a model that accurately segments clothes from images of people, as a component for virtual fitting rooms.

## 1. Dataset choice and reason

**Dataset:** `mattmdjaga/human_parsing_dataset` on the Hugging Face Hub — a copy of the **ATR (human parsing) dataset**, containing **17,706 image/mask pairs**.

Each mask is pixel-labeled with **18 classes**:

| ID | Label         | ID | Label      | ID | Label     |
|----|---------------|----|------------|----|-----------|
| 0  | Background    | 6  | Pants      | 12 | Left-leg  |
| 1  | Hat           | 7  | Dress      | 13 | Right-leg |
| 2  | Hair          | 8  | Belt       | 14 | Left-arm  |
| 3  | Sunglasses    | 9  | Left-shoe  | 15 | Right-arm |
| 4  | Upper-clothes | 10 | Right-shoe | 16 | Bag       |
| 5  | Skirt         | 11 | Face       | 17 | Scarf     |

### Why this dataset

- It is **person-centric with fine-grained clothing labels** (upper-clothes, pants, skirt, dress, belt, shoes, bag, scarf, hat, sunglasses) rather than a single generic "clothes" mask — this lets the model distinguish garment types, which is directly useful for a virtual fitting room (e.g., swapping only the "upper-clothes" region).
- It also labels body parts (face, arms, legs, hair) and background, which gives the model the context it needs to correctly draw the boundary between skin/hair and clothing — a common failure point in coarse clothes-only datasets.
- At ~17.7k images it is large enough to fine-tune a pretrained backbone without excessive overfitting, and it is directly accessible through the `datasets` library, which keeps preprocessing and reproducibility simple (no manual download/licensing friction).
- It is a well-established benchmark (the same data underlies the popular `segformer_b2_clothes` clothes-segmentation model), so results are comparable to existing published baselines.

### Comparison with other dataset options

ATR was chosen after weighing it against the other common human-parsing / fashion-segmentation datasets:

| Dataset                                          | Size          | Classes                  | Advantages                                                                                                                                                                                                      | Disadvantages                                                                                                                                                                               |
|--------------------------------------------------|---------------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ATR</b> (used here)                           | ~17.7K images | 18                       | Streams directly from the Hugging Face Hub with no manual setup; masks already aligned to a manageable, well-documented 18-class label set; single-person images keep the task simpler and faster to iterate on | Smaller than LIP/CIHP; mostly single-person, front-facing images, so it generalizes less well to occlusion, group photos, or unusual poses                                                  |
| <b>LIP</b> (Look Into Person)                    | ~50K images   | 20                       | Much larger and more diverse (pose, viewpoint, occlusion); a standard academic benchmark, so results are easy to compare against published work                                                                 | Heavier to download and preprocess; not directly streamable the way ATR is; longer training times for the same epoch budget                                                                 |
| <b>CIHP</b> (Crowd Instance-level Human Parsing) | ~38K images   | 20                       | Includes multi-person, crowded scenes with instance-level annotations — closer to real-world deployment conditions                                                                                              | Instance-level annotation adds pipeline complexity (needs instance segmentation, not just semantic parsing); heavier compute and longer training                                            |
| <b>Fashionpedia</b>                              | ~48K images   | 46 attributes/categories | Very fine-grained clothing attribute taxonomy (fabric, style, part-level detail), useful for fashion-attribute tasks specifically                                                                               | Not a pure body/clothing pixel-parsing format out of the box — built around bounding boxes + attributes, so masks need extra adaptation; far more classes than needed for this task's scope |
| <b>DeepFashion2</b>                              | ~490K images  | 13 clothing categories   | Massive scale, good for detection/retrieval-style clothing tasks                                                                                                                                                | Primarily bounding-box/landmark annotated, not full pixel-level body+clothing parsing; no body-part classes (hands, hair, face), which this task requires                                   |

The assignment scope (18-class body + clothing parsing on single-person images, trained within a limited time/compute budget) matched ATR's format and size directly — no reformatting was needed to go from raw dataset to trainable (image, mask) pairs. LIP or CIHP would likely improve generalization to harder real-world images (multi-person, occlusion) at the cost of significantly more preprocessing and training time; Fashionpedia and DeepFashion2 are better suited to attribute/detection-style fashion tasks than pixel-level body parsing and would have required non-trivial reformatting to use here.

**Split:** 80% train / 10% validation / 10% test, using a fixed seed ( 42 ) for reproducibility:

| Split      | Images |
|------------|--------|
| Train      | 14,164 |
| Validation | 1,771  |
| Test       | 1,771  |

Preprocessing

- Images converted to RGB, resized to 512×512 with bilinear interpolation, normalized to [0, 1].
- Masks resized to 512×512 with nearest-neighbor interpolation (to avoid inventing intermediate class IDs at label boundaries) and kept as integer class maps.

Data augmentation (training split only)

- Random horizontal flip (p = 0.5) — applied identically to image and mask.
- Random brightness jitter (max\_delta = 0.10).
- Random contrast jitter (range 0.9–1.1).
- Pixel values clipped back to [0, 1] after augmentation.

Augmentation intentionally avoids rotation/heavy geometric distortion, since the source images are near-frontal studio/e-commerce-style photos of people — the target deployment condition for a virtual fitting room — so aggressive geometric augmentation would train the model on scenarios it won't see in practice.

2. Model architecture

**DeepLabV3+ with a ResNet50 backbone** (pretrained on ImageNet), built with `tf.keras`.

Why DeepLabV3+

- Atrous/dilated convolutions (ASPP) capture multi-scale context without losing spatial resolution — important for segmenting both large regions (upper-clothes, pants) and thin/small ones (belt, scarf, sunglasses) in the same image.
- The decoder with a low-level skip connection (the "+" in DeepLabV3+) recovers the fine boundary detail that a plain DeepLabV3/ASPP-only head loses, which matters for clean garment edges — a "virtual fitting room" is far less useful if the clothing mask bleeds into skin or background.
- A ResNet50 ImageNet-pretrained backbone gives strong, general-purpose visual features from the start, which is critical given only ~14k fine-tuning images.

Architecture details

```
Input (512x512x3)
  |
  v
ResNet50 backbone (ImageNet weights)
  |-- low-level features <- conv2_block3_out (stride 4)
  \-- high-level features <- conv4_block6_out (stride 16)
      |
      v
ASPP block (on high-level features, 256 filters each branch):
  |-- 1x1 conv
  |-- 3x3 conv, dilation rate 6
  |-- 3x3 conv, dilation rate 12
  |-- 3x3 conv, dilation rate 18
  \-- global average pooling -> 1x1 conv -> upsample
    -> concat -> 1x1 conv -> BN -> ReLU -> Dropout(0.1)
        |
        v
Upsample ASPP output to low-level feature resolution
Low-level features -> 1x1 conv (48 filters) -> BN -> ReLU
  |
  v
Concatenate (ASPP output + projected low-level features)
  -> 3x3 conv -> BN -> ReLU (x2, 256 filters)
      |
      v
Upsample to 512x512
  |
  v
1x1 conv -> 18-channel softmax output (per-pixel class probabilities)
```

**Model size:** 17,834,578 total parameters (~68 MB); 17,799,794 trainable, 34,784 non-trainable (frozen BatchNorm stats from the pretrained stem).

Model experiments / alternatives compared

Two additional architectures were trained and evaluated under the same data split, preprocessing pipeline, and loss function, to validate the choice of DeepLabV3+/ResNet50:

| Model                               | Loss  | Pixel Accuracy | Mean IoU    |
|-------------------------------------|-------|----------------|-------------|
| DeepLabV3+ (ResNet50) — final model | 0.396 | 93.1%          | 0.513-0.534 |
| DeepLabV3+ (MobileNetV2 backbone)   | 0.45  | 91%            | 0.47        |
| U-Net                               | 0.62  | 89%            | 0.40        |

DeepLabV3+/ResNet50 was chosen as the final model based on this comparison. Beyond the better aggregate metrics, both the U-Net and the DeepLabV3+/MobileNetV2 variant were noticeably weaker on qualitative inference results — visibly noisier and less precise garment boundaries — not just on the numeric scores. The lighter MobileNetV2 backbone trades accuracy for a smaller, faster model, and U-Net's plain encoder-decoder (without ASPP's multi-scale context or the low-level skip fusion of DeepLabV3+) struggled more with the fine boundaries and small accessory classes (belt, scarf, shoes) that are central to this task.

### 3. Loss function selection and reason

The dataset is heavily class-imbalanced — background and skin/face pixels dominate, while accessories like belts, scarves, and individual shoes cover only a small fraction of pixels. A plain cross-entropy loss would let the model get a deceptively low loss by mostly getting the dominant classes right while ignoring rare ones. Two complementary losses were combined to address this:

#### a) Per-class weights (inverse-sqrt frequency)

Pixel counts per class are estimated from a random sample of 1,000 training masks. Class weights are computed as `1 / sqrt(frequency)`, then normalized so the average weight  $\approx 1$ . This is smoother than plain inverse-frequency (`1/frequency`), which tends to explode for extremely rare classes and destabilize training.

#### b) Weighted Focal Loss ( $\gamma = 2.0$ )

Down-weights the loss contribution of pixels the model already classifies confidently and correctly, focusing gradient updates on hard/misclassified pixels — exactly the issue with plain cross-entropy on an 18-class imbalanced mask, where a few dominant classes would otherwise dominate the gradient.

#### c) Weighted Multiclass Dice Loss

Dice loss operates directly on the overlap between predicted and ground-truth regions per class, which correlates closely with IoU (the assignment's evaluation metric) — optimizing it directly helps push mIoU up rather than only pixel-wise accuracy.

#### d) Combined loss

```
combined_loss = 0.5 x weighted_focal_loss + 0.5 x weighted_dice_loss
```

Dice is weighted more heavily than in an initial (unweighted, cross-entropy-based) version, since it correlates more directly with the mIoU target metric.

**Optimizer & schedule:** Adam, initial learning rate `1e-3`, with:

- `ReduceLROnPlateau` (factor 0.5, patience 3, monitoring `val_loss`)
- `EarlyStopping` (patience 7, restores best weights, monitoring `val_loss`)
- `ModelCheckpoint` saving the best model by `val_loss`

**Custom mean-IoU metric:** implemented manually via a confusion matrix per batch (`tf.math.confusion_matrix`), computing per-class IoU and averaging only over classes actually present in the batch's ground truth (to avoid penalizing/rewarding the score with undefined 0/0 unions).

### 4. Performance analysis and evaluation metrics

Trained for **6 epochs** on the training set (14,164 images), validated on 1,771 images.

#### Training curve (loss, per epoch)

| Epoch | Train loss | Train mIoU | Train acc. | Val loss | Val mIoU | Val acc. |
|-------|------------|------------|------------|----------|----------|----------|
| 1     | 0.4375     | 0.367      | 88.3%      | 0.4375   | 0.386    | 89.1%    |
| 2     | 0.4126     | 0.426      | 90.8%      | 0.5065   | 0.253    | 57.6%    |
| 3     | 0.4039     | 0.454      | 91.7%      | 0.4211   | 0.401    | 87.7%    |
| 4     | 0.3980     | 0.474      | 92.2%      | 0.4344   | 0.426    | 87.7%    |
| 5     | 0.3934     | 0.492      | 92.7%      | 0.4304   | 0.415    | 90.7%    |

|   |        |       |       |        |       |       |
|---|--------|-------|-------|--------|-------|-------|
| 6 | 0.3892 | 0.509 | 93.2% | 0.4015 | 0.510 | 92.7% |
|---|--------|-------|-------|--------|-------|-------|

Validation was noisy epoch-to-epoch (notably a dip at epoch 2), typical of a small validation set (1,771 images) combined with a relatively high initial learning rate; `ReduceLROnPlateau` / `EarlyStopping` were configured specifically to guard against this, and training loss decreased monotonically throughout. The best checkpoint (epoch 6, lowest `val_loss`) was restored automatically for evaluation.

Test-set results (1,771 held-out images)

| Metric                                                     | Value  |
|------------------------------------------------------------|--------|
| Loss (combined focal + Dice)                               | 0.3964 |
| Pixel accuracy                                             | 93.09% |
| Mean IoU (Keras metric, averaged per batch)                | 0.5133 |
| Mean IoU (per-class, computed once over the full test set) | 0.5336 |

Per-class IoU (full test set)

| Class         | IoU   | Class      | IoU   | Class     | IoU   |
|---------------|-------|------------|-------|-----------|-------|
| Background    | 0.974 | Belt       | 0.346 | Left-arm  | 0.261 |
| Hat           | 0.672 | Left-shoe  | 0.289 | Right-arm | 0.382 |
| Hair          | 0.774 | Right-shoe | 0.286 | Bag       | 0.634 |
| Sunglasses    | 0.624 | Face       | 0.820 | Scarf     | 0.351 |
| Upper-clothes | 0.718 | Left-leg   | 0.398 |           |       |
| Skirt         | 0.517 | Right-leg  | 0.370 |           |       |
| Pants         | 0.735 |            |       |           |       |
| Dress         | 0.456 |            |       |           |       |

Clothing-item classes only (upper-clothes, skirt, pants, dress, belt, hat, sunglasses, bag, scarf) average  $\approx 0.561$  mIoU — noticeably better than the body-limb classes, which is consistent with the assignment's focus on garments rather than fine limb segmentation.

5. Discover system capabilities

Strengths

- Large, well-defined garment regions are segmented reliably: Pants (0.735), Upper-clothes (0.718), and Hair (0.774) all score well above the overall mean, and Face (0.820) and Background (0.974) are segmented almost perfectly.
- High overall pixel accuracy (93.1%), confirming the model correctly labels the great majority of pixels even where region-level IoU for small classes is lower.
- The ASPP module's multi-scale context handles a mix of poses and framing (full-body, half-body) reasonably well without retraining per framing type.
- Bag detection (0.634 IoU) is notably strong for an accessory class that occupies a small, variably-shaped image region — likely because bags have high visual contrast against clothing/background.
- Inference is a single forward pass (no post-processing/CRF needed), making the pipeline simple to productionize behind an image-upload API.

Drawbacks / weaknesses

- Small, thin, or symmetric-pair classes underperform: Left/Right shoe ( $\sim 0.29$ ), Belt (0.346), Scarf (0.351), and Left/Right arm (0.26–0.38) all score well below the mean. These regions are small in pixel count, easily confused with their left/right counterpart, and often partially occluded by other clothing.
- Dress vs. Skirt/Upper-clothes confusion: Dress (0.456) likely gets confused with the combination of skirt + upper-clothes, since a dress visually spans both regions and the boundary between "dress" and "upper-clothes + skirt" is inherently ambiguous even for human annotators.
- Validation instability during training (epoch 2's dip) suggests the model is still somewhat sensitive to the learning-rate/batch-size combination and would benefit from more epochs and/or a lower initial learning rate for a production model.
- Only trained for 6 epochs — the training loss was still decreasing at the final epoch, suggesting the model has not fully converged and additional training would likely improve mIoU further, especially for the weaker minority classes.
- Training was capped by dataset size and available compute resources: with  $\sim 17.7k$  images at  $512 \times 512$  resolution and a ResNet50-backed DeepLabV3+ model, each epoch is computationally expensive, and limited compute/time resources meant training could not be extended much beyond 6 epochs. This was a resource constraint rather than a modeling choice, and is the main reason the model has not yet fully converged.
- The custom mean-IoU Keras metric (batch-averaged) and the offline per-class mean-IoU differ (0.513 vs 0.534) because the former is averaged across mini-batches (each excluding absent classes per batch) while the latter is computed once over the full test set — the

offline number is the more reliable estimate and should be used as the reported metric.

### Limitations of the system (deployment conditions)

- **Framing & scene:** best suited to single-person, near-frontal, evenly-lit photos (e-commerce/fitting-room style). Multi-person scenes, heavy occlusion, or overlapping garments (e.g. a jacket over a dress) are outside what the model was trained for, since each pixel only gets one class label.
- **Image resolution/quality:** input is always resized to 512×512, so both very high-resolution and low-resolution/compressed images lose detail after resizing — this hits thin accessories (belt, scarf, shoes) hardest and matches their lower IoU scores.
- **Small classes:** shoes, belts, scarves, and sunglasses are the most error-prone in practice, not just in the IoU numbers — they're often absorbed into a neighboring class (e.g. a thin belt merged into "pants"). Not yet reliable enough for automated garment-swap use cases without extra targeted data/augmentation.
- **Gender imbalance:** ATR leans heavily toward women's clothing (dresses, skirts), with few menswear examples, so the model is expected to be less reliable on men's clothing — a dataset bias to fix with more balanced data, not an architecture issue.
- **Out-of-distribution inputs:** filtered/stylized photos (beauty filters, heavy color grading, AR overlays) and casual "in the wild" phone photos differ from ATR's studio-style training images and haven't been separately validated — expect lower quality on these.

## 6. Suggested next steps

- Train for more epochs (validation loss was still improving at epoch 6) and/or lower the initial learning rate to reduce the mid-training validation instability.
- Add targeted augmentation (crops around limbs/shoes, mild rotation) to lift the weakest classes (shoes, belt, scarf, arms).
- Evaluate boundary-quality metrics (e.g., boundary F-score) in addition to mIoU, since garment edges matter more than raw IoU for a virtual-fitting-room use case.
- Consider merging near-duplicate/ambiguous classes (e.g., dress vs. upper-clothes+skirt) if the downstream application only needs a coarser "clothing vs. non-clothing vs. body" mask.
- Try additional backbones/architectures beyond the three compared here (Section 2) — e.g. a Transformer-based segmentation head (SegFormer) — now that DeepLabV3+/ResNet50 has established a solid baseline to beat.